JWT (JSON Web Token)

Принципы работы, применение и вопросы безопасности

План доклада

- Проблема аутентификации
- Определение JWT
- Структура токена
- Механизм аутентификации
- Алгоритмы подписи
- Преимущества и ограничения
- Вопросы безопасности
- Access и Refresh токены
- Области применения
- Выводы

Проблема: идентификация пользователя

- HTTP — протокол **без состояния** (stateless)
- Классический подход: серверные сессии и cookie с `session_id`
- Ограничения сессионного подхода:
  - необходимость хранения состояния на сервере
  - сложность горизонтального масштабирования
  - неудобство при работе с мобильными клиентами и микросервисами

> **Альтернативный подход:** самодостаточные токены, не требующие хранения на сервере — JWT

Определение JWT

JSON Web Token (RFC 7519) — открытый стандарт передачи данных между сторонами в формате JSON.

- Компактное и самодостаточное представление (self-contained)
- Поддерживает подпись и/или шифрование
- Основная область применения: **аутентификация и авторизация**

`xxxxx . yyyyy . zzzzz`

Header · Payload · Signature

Структура: Header

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

- `alg` — алгоритм подписи (HS256, RS256 и др.)
- `typ` — тип токена
- Кодируется в **Base64Url** — это кодирование, а не шифрование

Структура: Payload (claims)

```
{
  "sub":  "1234567890",
  "name": "Ivan Ivanov",
  "role": "admin",
  "iat":  1516239022,
  "exp":  1516242622
}
```

Категории claims

- **Registered:** iss, sub, aud, exp, nbf, iat, jti
- **Public:** пользовательские, но стандартизованные
- **Private:** согласованные между сторонами

Важное замечание

Payload доступен для чтения любой стороной — не следует размещать конфиденциальные данные.

Структура: Signature

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  secret
)
```

- Обеспечивает **целостность**: данные не могут быть изменены незаметно
- Секретный ключ известен только серверу (для HS256)
- При недействительной подписи токен **отклоняется**

> Для наглядного анализа токенов удобно использовать сервис **jwt.io**.

Механизм аутентификации




```
Authorization: Bearer <token>
```

Сервер проверяет подпись и срок действия **без обращения к базе данных**

Алгоритмы подписи

HS256 — симметричный

- HMAC на основе SHA-256, единый секрет
- Высокая производительность и простота
- Секрет известен всем проверяющим сторонам

RS256 / ES256 — асимметричный

- Приватный ключ — для подписи
- Публичный ключ — для проверки
- Удобно для микросервисной архитектуры

Режим **alg: none** (без подписи) недопустим в промышленной эксплуатации.

Преимущества JWT

- **Stateless** — отсутствие необходимости хранить состояние на сервере
- Простота горизонтального масштабирования
- Компактность — применим в URL, заголовках, cookie
- Кроссплатформенность и поддержка кросс-доменных сценариев
- Самодостаточность — данные о субъекте содержатся в токене
- Наличие стандарта и зрелых библиотек

Ограничения JWT

- Невозможность отзыва до истечения срока exp без дополнительных механизмов
- Payload не зашифрован — риск утечки при XSS
- Размер превышает традиционный идентификатор сессии
- Затруднено динамическое изменение прав доступа
- Ошибки реализации приводят к уязвимостям
- Не предназначен для хранения конфиденциальных данных

Безопасность: типичные ошибки

- Приём токенов с **alg: none**
- **Слабый секретный ключ** (рекомендуется не менее 256 бит)
- **Подмена алгоритма** HS256 / RS256
- Хранение в **localStorage** — уязвимость к XSS

- Избыточно **длительный exp**
- Размещение **конфиденциальных данных** в payload
- Отсутствие проверки **aud** и **iss**
- Отсутствие **ротации refresh-токенов**

Рекомендуется хранить токены в **httpOnly + Secure cookie**.

Access и Refresh токены

Подход, решающий проблему отзыва и снижающий риски компрометации:

Access token

- Короткий срок жизни: 5–15 минут
- Используется для доступа к API
- Ограниченный ущерб при утечке

Refresh token

- Длительный срок: дни или недели
- Хранится на сервере → может быть отозван
- Применяется только для обновления access-токена

Области применения JWT

Целесообразно

- Микросервисные архитектуры и API
- Мобильные приложения и SPA
- Протоколы OAuth2 и OpenID Connect
- Обмен данными между доверенными сервисами

Нецелесообразно

- Классические серверные веб-приложения с сессиями
- Сценарии, требующие мгновенного отзыва доступа
- Передача больших объёмов данных
- Использование в качестве замены шифрованию

Выводы

- JWT — стандартизованный формат токенов на основе JSON
- Структура: **Header.Payload.Signature**
- Подпись обеспечивает целостность, но не конфиденциальность
- Отсутствие состояния упрощает масштабирование API и микросервисов
- Ключевые риски: невозможность отзыва, XSS, слабые ключи, атаки на алгоритм
- Рекомендуемая практика: короткие access-токены, refresh-токены, httpOnly cookie, надёжные алгоритмы подписи

Благодарим за внимание